

Table des matières

Labo 08 OLAP — Données semi-structurées sur l'entrepôt	1
Le terrain, en une minute	1
Partie 1 — Fabriquer le document.....	2
Partie 2 — L'aller-retour par un fichier.....	3
Partie 3 — Interroger, typer, aplatir	4
Partie 4 — Ce que le contraste vous a montré	8
Nettoyage.....	9
Et ensuite ?.....	9

Labo 08 OLAP — Données semi-structurées sur l'entrepôt

Laboratoire du Cours 08, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

Ce laboratoire fait un aller-retour complet : on fabrique un document JSON à partir des tables de l'entrepôt, on l'écrit dans un fichier, on le recharge comme s'il venait d'une API, puis on l'interroge et on l'aplatit. Tout se passe dans la ligne de commande DuckDB, ouverte depuis `Projet_SQL/` sur l'entrepôt, et les fichiers écrits atterrissent dans ce dossier. Comptez une heure.

Les deux voies — Ce labo a un jumeau, `Labo_08_OLTP.md`, qui fait le même parcours sur PostgreSQL. Les deux moteurs partagent les flèches `->` et `->>`, et presque tous les pièges ; ils divergent sur le chargement des fichiers et sur l'index. Gardez les deux fichiers ouverts côte à côte.

Pourquoi on fabrique la donnée — AdventureWorks DW ne contient aucun JSON, et la formation refuse d'ajouter un téléchargement de plus. La parade est meilleure que le problème : en construisant le document vous-même, vous comprenez sa forme avant de l'interroger, ce qu'un fichier tombé du ciel ne vous apprendrait pas.

Le terrain, en une minute

USE dbo;

Deux tables suffisent : `FactInternetSales` pour les lignes de commande, `DimCustomer` pour le client. On créera deux tables de travail, préfixées `LAB008_`, et deux fichiers JSON, `labo08_commandes.json` et `labo08_tableau.json`, tous faciles à supprimer à la fin. Les fonctions JSON viennent de l'extension `json` de DuckDB, livrée avec la ligne de commande : rien à installer.

Partie 1 — Fabriquer le document

1.1 Un premier objet

Théorie : Theorie_08.md, section 3.

```
SELECT JSON_OBJECT('produit', f.ProductKey,  
                  'qte',    f.OrderQuantity,  
                  'montant', f.SalesAmount) AS ligne  
FROM   FactInternetSales AS f  
LIMIT 3;
```

Attendu : trois documents à trois clés, de type JSON, chacun sur une ligne, comme {"produit":310,"qte":1,"montant":3578.2700}. Le montant garde ses quatre décimales, celles du DECIMAL(19,4) de l'entrepôt.

1.2 Un document imbriqué

Théorie : Theorie_08.md, section 3. Théorie : Theorie_03.md, section 2.

Une commande porte plusieurs lignes. On les rassemble en tableau avec JSON_GROUP_ARRAY, l'agrégat qui fait d'un groupe de valeurs un tableau JSON, puis on emboîte le tout dans un objet.

```
CREATE OR REPLACE TABLE LABO08_COMMANDE AS  
SELECT f.SalesOrderNumber AS numero,  
       JSON_OBJECT(  
         'commande', f.SalesOrderNumber,  
         'client',   f.CustomerKey,  
         'date',     f.OrderDateKey,  
         'lignes',   JSON_GROUP_ARRAY(JSON_OBJECT('produit', f.ProductKey,  
                                                  'qte',    f.OrderQuantity,  
                                                  'montant', f.SalesAmount))  
       ) AS document  
FROM   FactInternetSales AS f  
GROUP BY f.SalesOrderNumber, f.CustomerKey, f.OrderDateKey;  
  
SELECT COUNT(*) FROM LABO08_COMMANDE;  
SELECT document FROM LABO08_COMMANDE WHERE numero = 'SO51178';
```

Attendu : 27 659 commandes, autant que de numéros distincts dans FactInternetSales, et un document dont la clé lignes contient un tableau de trois objets, un par ligne de la commande SO51178.

1.3 Le piège de la clé nulle

Théorie : Theorie_08.md, section 3.

```
SELECT JSON_OBJECT('a', 1, 'b', NULL) AS avec_null;
```

Attendu : {"a":1,"b":null}, les deux clés. DuckDB garde la clé et lui donne la valeur null de JSON, comme PostgreSQL dans le labo jumeau. Snowflake, d'après sa documentation, fait l'inverse : son OBJECT_CONSTRUCT écarte la clé, et seul OBJECT_CONSTRUCT_KEEP_NULL la garde. Retenez cet écart : deux documents fabriqués à partir de la même ligne n'ont pas les mêmes clés d'un moteur à l'autre, et un test de présence de clé ne donne pas le même résultat. La sous-partie 3.1 montrera que ce null n'est pas non plus le NULL de SQL.

Partie 2 — L'aller-retour par un fichier

C'est ce qui rend l'exercice réaliste : on sort le document de la base, puis on le fait rentrer comme s'il arrivait du dehors.

2.1 Écrire les documents dans un fichier

Théorie : Theorie_08.md, section 7.

```
COPY (SELECT document FROM LAB008_COMMANDE ORDER BY numero)
TO 'labo08_commandes.json';
```

Attendu : un fichier labo08_commandes.json d'un peu plus de 4 Mo dans Projet_SQL/, une commande par ligne. DuckDB choisit le format d'après l'extension, et écrit chaque ligne de la requête comme un objet dont les clés sont les noms des colonnes : notre colonne document devient une enveloppe, {"document":{"commande":"S043697",...}}. Ouvrez le fichier dans un éditeur de texte pour le voir.

2.2 Recharger en brut

Théorie : Theorie_08.md, section 7.

```
SELECT json FROM READ_JSON_OBJECTS('labo08_commandes.json') LIMIT 1;
```

```
CREATE OR REPLACE TABLE LAB008_BRUT AS
SELECT json -> 'document' AS charge
FROM READ_JSON_OBJECTS('labo08_commandes.json');
```

```
SELECT COUNT(*) FROM LAB008_BRUT;
SELECT column_name, column_type FROM (DESCRIBE LAB008_BRUT);
```

Attendu : READ_JSON_OBJECTS rend chaque objet du fichier tel quel, dans une colonne nommée json, de type JSON. La flèche -> descend d'un cran dans le document et déballé l'enveloppe au passage ; la partie 3 en fait le tour. Puis 27 659 lignes, une par document, dans une table d'accueil à une seule colonne,

charge, de type JSON. Vous venez de reproduire, en petit, ce que fait un pipeline d'ingestion : une table dont personne n'a eu besoin de connaître le schéma à l'avance.

Astuce — `READ_JSON`, sans `_OBJECTS`, fait autre chose : il devine un schéma et rend des colonnes typées, ici une structure document dont le montant sort en `DOUBLE`, un type approché, alors que la source était en `DECIMAL(19,4)`. Pour accueillir un document tel quel, `READ_JSON_OBJECTS` ; pour obtenir des colonnes, déclarez leur type plutôt que de le laisser deviner.

Portabilité — Sur Snowflake, le même aller-retour passe par un stage, une zone de fichiers rattachée au compte : `COPY INTO @stage` pour écrire, `COPY INTO table` pour relire, dans une table dont la seule colonne est un `VARIANT`.

2.3 Le fichier qui ne contient qu'un tableau

Théorie : Theorie_08.md, section 7.

Refaites l'écriture, mais en un seul tableau JSON qui contient cent commandes, la forme `[{...},{...}]` que rendent beaucoup d'API :

```
COPY (SELECT document FROM LABO08_COMMANDE ORDER BY numero LIMIT 100)
TO 'labo08_tableau.json' (ARRAY true);

SELECT COUNT(*) AS sans_rien_dire FROM READ_JSON_OBJECTS('labo08_tableau.json');

SELECT COUNT(*) AS non_structure
FROM READ_JSON_OBJECTS('labo08_tableau.json', format = 'unstructured');

SELECT COUNT(*) AS en_tableau
FROM READ_JSON_OBJECTS('labo08_tableau.json', format = 'array');
```

Attendu : 100 sans rien préciser, 1 en `unstructured`, 100 en `array`. Le format décide de tout : lu comme un seul document, le fichier donne une seule ligne, qui contient le tableau entier ; lu comme un tableau, il donne une ligne par élément. Sans l'option, DuckDB devine, et devine juste ici. Le piège est d'imposer le mauvais format, comme le fait un pipeline réglé pour un document par ligne :

```
SELECT COUNT(*) AS une_par_ligne
FROM READ_JSON_OBJECTS('labo08_tableau.json', format = 'newline_delimited');
```

Attendu : `Malformed JSON in file "labo08_tableau.json", at byte 2 in line 2: unexpected end of data`. Celui-là prévient ; le `unstructured` de tout à l'heure, lui, rendait sa ligne unique sans rien dire. Sur Snowflake, c'est l'option `STRIP_OUTER_ARRAY` qui tient ce rôle, et elle est la plus souvent oubliée d'un premier chargement JSON.

Partie 3 — Interroger, typer, aplatir

3.1 Extraire, et le piège des guillemets

Théorie : *Theorie_08.md*, section 4.

```
SELECT charge -> 'commande'          AS en_json,  
       charge ->> 'commande'         AS en_texte,  
       charge -> 'client'            AS client_json,  
       (charge ->> 'client')::INTEGER AS client_entier  
FROM   LAB008_BRUT  
LIMIT 5;
```

Regardez la première colonne : la valeur porte ses guillemets, "S043697", et son type est JSON. La flèche simple rend un morceau de document, la flèche double rend du texte nu, en VARCHAR. La règle est celle de PostgreSQL : flèche simple tant qu'on descend, flèche double au dernier cran. Maintenant l'expérience qui compte :

-- Les commandes numérotées S0437..., version fautive.

```
SELECT COUNT(*) AS sans_conversion  
FROM   LAB008_BRUT  
WHERE  (charge -> 'commande') LIKE 'S0437%';
```

-- Version correcte.

```
SELECT COUNT(*) AS avec_conversion  
FROM   LAB008_BRUT  
WHERE  (charge ->> 'commande') LIKE 'S0437%';
```

Attendu : zéro dans le premier cas, cent dans le second. Aucune erreur, aucun avertissement : la première requête compare "S043701", guillemets compris, au motif, et rien ne commence par S0437 quand le premier caractère est un guillemet. Un IN sur une sous-requête se trompe de la même façon. C'est le même mode de défaillance que le NOT IN du Cours 07, et vous devriez maintenant le reconnaître de loin. L'égalité, curieusement, protège mieux :

```
SELECT COUNT(*) FROM LAB008_BRUT WHERE charge -> 'commande' = 'S043701';  
SELECT COUNT(*) FROM LAB008_BRUT WHERE (charge -> 'commande') = 'S043701';
```

Attendu : deux erreurs, et c'est une chance. La première vient des parenthèses absentes : DuckDB lie -> moins fort que =, lit charge -> ('commande' = 'S043701'), et répond No function matches the given name and argument types 'json_extract(JSON, BOOLEAN)', d'où les parenthèses des requêtes précédentes. La seconde tente de convertir 'S043701' en JSON pour le comparer, et échoue : Malformed JSON at byte 0 of input. Restent la clé à null de la sous-partie 1.3, et le troisième piège silencieux du module :

```
SELECT (JSON_OBJECT('a', 1, 'b', NULL) -> 'b') IS NULL AS fleche_simple,  
       (JSON_OBJECT('a', 1, 'b', NULL) ->> 'b') IS NULL AS fleche_double,  
       (JSON_OBJECT('a', 1, 'b', NULL) -> 'c') IS NULL AS cle_absente;
```

Attendu : false, true, true. Le null de JSON est une valeur, pas l'absence de valeur : la flèche simple le rend comme un morceau de document, et IS NULL répond faux. La flèche double le convertit en NULL, comme une clé absente. Un filtre -> ... IS NULL, écrit pour trouver les valeurs manquantes, rate donc toutes celles qui sont présentes à null, sans erreur.

3.2 Le tri alphabétique déguisé

Théorie : Theorie_08.md, section 5. Théorie : Theorie_02.md, section 4.

```
-- Sans conversion : Le tri est ALPHABÉTIQUE.
SELECT charge ->> 'commande' AS numero,
       charge -> 'lignes' -> 0 -> 'montant' AS premier_montant
FROM   LAB008_BRUT
ORDER BY charge -> 'lignes' -> 0 -> 'montant' DESC, numero
LIMIT 5;

-- Avec conversion : Le tri est numérique.
SELECT charge ->> 'commande' AS numero,
       (charge -> 'lignes' -> 0 ->> 'montant')::DECIMAL(19,4) AS premier_montant
FROM   LAB008_BRUT
ORDER BY premier_montant DESC, numero
LIMIT 5;
```

Comparez les deux listes. La première s'ouvre sur des montants de 9,99 \$, la seconde sur 3 578,27 \$. Un JSON se trie comme du texte, caractère par caractère, et le caractère 9 vient après le 3 : la liste a l'air triée, elle est fausse. -> 0 descend dans le tableau par la position, comptée à partir de 0. La conversion se fait en DECIMAL(19,4), le type de SalesAmount dans l'entrepôt ; la sous-partie 3.5 montrera ce que coûte un type arrondi.

3.3 La conversion tolérante

Théorie : Theorie_08.md, section 5.

```
SELECT TRY_CAST('123.45' AS DECIMAL(19,4)) AS bonne_valeur,
       TRY_CAST('n/a' AS DECIMAL(19,4)) AS valeur_invalide;
```

Attendu : la valeur, puis NULL. Comparez avec la conversion ordinaire :

```
SELECT CAST('n/a' AS DECIMAL(19,4));
```

Attendu : Could not convert string "n/a" to DECIMAL(19,4), et la requête s'arrête. Sur une source dont vous ne maîtrisez pas la qualité, la seconde forme transforme une ligne fautive en panne de tout le lot.

3.4 Aplatir le tableau des lignes

Théorie : Theorie_08.md, section 6.

```

SELECT c.charge ->> 'commande' AS commande,
       ligne.key::INTEGER AS position,
       (ligne.value ->> 'produit')::INTEGER AS produit,
       (ligne.value ->> 'qte')::INTEGER AS qte,
       (ligne.value ->> 'montant')::DECIMAL(19,4) AS montant
FROM LABO08_BRUT AS c,
     JSON_EACH(c.charge -> 'lignes') AS ligne
WHERE c.charge ->> 'commande' = 'SO51178'
ORDER BY position;

```

Attendu : les trois lignes de la commande SO51178, aux positions 0, 1 et 2. JSON_EACH rend une ligne par élément du tableau, avec sa clé et sa valeur ; pour un tableau, la clé est la position, rendue en texte, d'où le ::INTEGER qui la fait trier comme un nombre. Placée après la table dans le FROM, la fonction voit chaque document tour à tour : DuckDB rend le LATERAL implicite. Vous êtes revenu au relationnel. Vérifiez-le en comptant :

```

-- Le nombre de lignes aplaties doit éгалer la somme des tailles de tableau.
SELECT COUNT(*) AS lignes_aplaties
FROM LABO08_BRUT AS c, JSON_EACH(c.charge -> 'lignes') AS ligne;

SELECT SUM(JSON_ARRAY_LENGTH(charge -> 'lignes')) AS somme_des_tailles FROM LABO08_BRUT;

```

Attendu : 60 398 des deux côtés, le nombre de lignes de FactInternetSales : l'aplatissement a retrouvé chaque ligne de commande. Testez maintenant un document dont le tableau est vide :

```

SELECT d.id, ligne.key, ligne.value
FROM (SELECT 1 AS id, '{"lignes":[]}'::JSON AS doc
      UNION ALL
      SELECT 2, '{"lignes":[{"produit":1}]}'::JSON) AS d
LEFT JOIN LATERAL JSON_EACH(d.doc -> 'lignes') AS ligne ON true
ORDER BY d.id;

```

Attendu : deux lignes, dont celle du document 1, avec des NULL dans les colonnes produites par JSON_EACH. Avec une virgule à la place du LEFT JOIN LATERAL ... ON true, elle disparaîtrait. C'est le LEFT JOIN LATERAL du Cours 07, et l'option OUTER => TRUE du FLATTEN de Snowflake.

3.5 La comparaison qui conclut le module

Théorie : Theorie_08.md, section 9.

Posez la même question de deux façons : le chiffre d'affaires par année, une fois depuis le document, une fois depuis la table relationnelle d'origine. .timer on fait afficher le temps de chaque requête, et // est la division entière de DuckDB : 20101229 // 10000 donne 2010.

```

.timer on

-- Depuis le document.

```

```

SELECT (c.charge ->> 'date')::INTEGER // 10000 AS annee,
       SUM((ligne.value ->> 'montant')::DECIMAL(19,4)) AS total
FROM   LAB008_BRUT AS c, JSON_EACH(c.charge -> 'lignes') AS ligne
GROUP BY annee
ORDER BY annee;

-- Depuis Le relationnel.
SELECT f.OrderDateKey // 10000 AS annee, SUM(f.SalesAmount) AS total
FROM   FactInternetSales AS f
GROUP BY annee
ORDER BY annee;

.timer off

```

Attendu : les mêmes cinq années, de 2010 à 2014, et les mêmes totaux au dix-millième près, dont 16 351 550,3400 \$ pour 2013. Mais pas le même effort : sur notre poste, la version document prend environ deux dixièmes de seconde, et la relationnelle moins d’un centième, une trentaine de fois moins. Chaque requête relit et découpe chaque document.

Les totaux coïncident parce que le montant a été converti en DECIMAL (19,4), le type de la source. Converti en DECIMAL (18,2), comme on serait tenté de le faire pour des dollars, il perdrait ses deux dernières décimales sur 2 449 lignes : autant de commandes changeraient de total de quelques cents, et le chiffre d’affaires de l’ensemble s’écarterait de 0,67 \$ de celui de l’entrepôt.

La question à trancher — Laquelle de ces deux écritures garderiez-vous en production, et pourquoi ? Écrivez votre réponse avant de lire la suivante. Un bon raisonnement pèse trois choses : le coût de lecture pour l’humain, le coût d’exécution pour la machine, et ce qui arrive le jour où le fournisseur ajoute un champ.

Partie 4 — Ce que le contraste vous a montré

Théorie : Theorie_08.md, section 10.

Vous verrez la colonne PostgreSQL se remplir en faisant Labo_08_OLTP.md. Notez déjà ce que l’entrepôt vous a appris.

Observation	Sur l’entrepôt, DuckDB
Le type	JSON, un texte validé à l’entrée, qui se trie comme du texte
Le chargement	READ_JSON_OBJECTS, et l’option format qui décide du sort d’un fichier [{...},{...}]

Observation	Sur l'entrepôt, DuckDB
L'aller-retour	COPY ... TO écrit le fichier, chaque document enveloppé sous le nom de sa colonne
Le piège des guillemets	l'égalité avec une chaîne échoue, mais LIKE et IN comparent les guillemets et rendent zéro, sans erreur
La clé nulle	gardée, avec la valeur null de JSON, que -> ... IS NULL ne voit pas
L'aplatissement	JSON_EACH, qui rend aussi la position, en texte
La performance	aucun index ; chaque requête relit les documents, une trentaine de fois plus lentement que le relationnel ici

Nettoyage

DROP TABLE IF EXISTS supprime la table sans se plaindre si elle a déjà disparu (Cours 03, section 7).

```
DROP TABLE IF EXISTS LABO08_COMMANDE;  
DROP TABLE IF EXISTS LABO08_BRUT;
```

Les deux fichiers, labo08_commandes.json et labo08_tableau.json, restent dans Projet_SQL/ : supprimez-les à la main, ils ne servent plus.

Et ensuite ?

Vous savez faire entrer une donnée dont vous ne décidez pas la forme, et la ramener au relationnel. Le JSON_EACH que vous venez de placer après la table est le même outil que le LATERAL du Cours 07, appliqué à un tableau au lieu d'une table.

Le Cours 09 réemploiera ce LATERAL une troisième fois, pour l'as-of join : « la valeur en vigueur au moment de l'événement ». Trois usages, une seule idée.

Avant d'y aller, les exercices dans Exercices_corrige_08.md.